In [6]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

In [ ]:
```
 cd drive/MyDrive/Colab\ Notebooks
```

cd drive/MyDrive/Colab\ Notebooks

In [ ]:
```
!jupyter nbconvert --to html cap_3.ipynb
```

GEOPROCESSAMENTO E GERAR PONTOS AMOSTRAIS

In [ ]:
```
# Importar bibliotecas
import geopandas as gpd

# Carregar o polígono
poligono_BJ = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\600m.shp")

# Reprojetar para UTM Zona 24S (EPSG:32724)
poligono_BJ = poligono_BJ.to_crs(epsg=32724)

# Calcular a área em km²
area_poligono_BJ = poligono_BJ.geometry.area / 1e6  # Convertendo de m² para km²

print(area_poligono_BJ)
poligono_BJ.explore()
```

In [ ]:
```
# Importar bibliotecas
import geopandas as gpd

# Carregar os polígonos
poligono_geral = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\superficie_prev.shp")
poligono_BJ = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\600m.shp")

# Reprojetar para UTM Zona 24S (EPSG:32724)
poligono_geral = poligono_geral.to_crs(epsg=32724)
poligono_BJ = poligono_BJ.to_crs(epsg=32724)

# Subtrair polígono_BJ do polígono_geral
area_amostral = poligono_geral.geometry.difference(poligono_BJ.geometry)

# Criar um GeoDataFrame com CRS
area_amostral = gpd.GeoDataFrame(geometry=area_amostral, crs="EPSG:32724")

# Calcular a área em km²
area_area_amostral = area_amostral.geometry.area / 1e6  # Convertendo de m² para km²

print(area_area_amostral)
area_amostral.explore()
```

In [ ]:
```
#Importar bibliotecas
import geopandas as gpd
import geodatasets
#!pip install folium matplotlib mapclassify #Usado para instalar o pacote Folium matplotlib

#Polígonos
polygons = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\600m.shp");

#Gerar pontos amostrais aleatoriamente em cada polígono
#n200_sampled_points = polygons.sample_points(2000)
m = polygons.explore()

#Individualizar cada ponto usando o método 'Explode'
n200_sampled_points_indiv = n200_sampled_points.explode(index_parts = False).reset_index(drop = True)

#Exportar geodataframe como shp
n200_sampled_points_indiv.to_file('C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\pontos_amostrais_600m.shp')
pontos_amos = gpd.read_file('C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\pontos_amostrais_600m.shp')
pontos_amos.explore()
```

GERAR PONTOS AMOSTRAIS FORA DOS BREJOS

In [ ]:
```
#Polígono geral
poligono_geral = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\superficie_prev.shp");

#Polígono 600m
poligono_BJ = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\600m.shp");

#Subtrair brejos do polígono geral
area_amostral = poligono_geral['geometry'].difference(poligono_BJ)

#Gerar pontos amostrais aleatoriamente no polígono
a200_sampled_points = area_amostral.sample_points(3000)

#Individualizar cada ponto usando o método 'Explode'
a200_sampled_points_indiv = a200_sampled_points.explode(index_parts = False).reset_index(drop = True)
```

```
#Exportar geodataframe como shp
a200_sampled_points_indiv.to_file('C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\pontos_amostrais_superficie.shp')

#Visualizar
a200_sampled_points_indiv.explore()
```

In [ ]:
```
bj = 51337.529052
ex = 399182.814141
area_total = ex + bj
area_bj_porcento = (bj/area_total)*100

print(area_total)
print(area_bj_porcento)
```

```
450520.34319299995
11.39516335447861
```

TRATAMENTO DOS DADOS E ANÁLISE ESTATÍSTICA - ESTIMATIVA DE CO2

DADOS DE ESTIMATIVA DE FLUXO DE CO2 - 600M

In [ ]:
```
#Importar bibliotecas
import pandas as pd
import geopandas as gpd
import os

#Importar dados das amostras de co2flux
#Criar variáveis referentes a cada planilha
co2_BJ = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ\\co2_BJ.csv", sep = ',', encoding = 'utf-8')

#Renomear coluna
co2_BJ = co2_BJ.rename(columns ={'first': 'co2flux_600m'})

#Selecionar apenas a coluna com valores das amostras de co2flux
co2_BJ_val = co2_BJ['co2flux_600m']

#Coverter coluna para dataframe
co2_BJ_val = pd.DataFrame(co2_BJ_val)

#Excluir valores NaN (nulos)
co2_BJ_val = co2_BJ_val.dropna()

#Análise de valores missing
co2_BJ_val.isnull().sum()
```

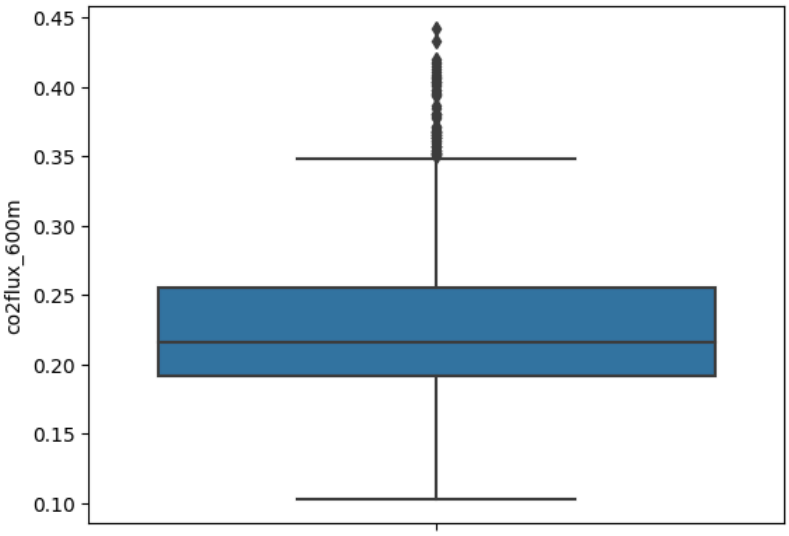
Out[ ]:
```
co2flux_600m    0
dtype: int64
```

In [ ]:
```
co2_BJ_val.describe()
```

Out[ ]:

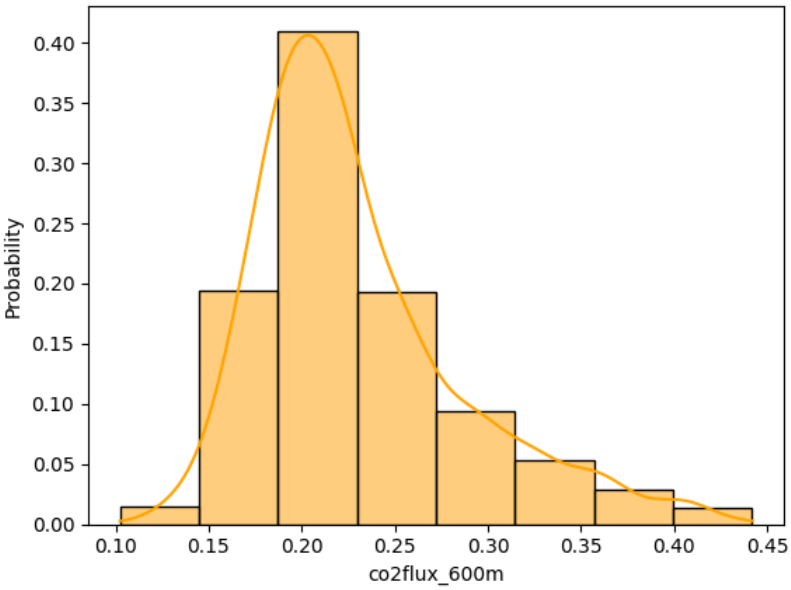
	co2flux_600m
count	1419.000000
mean	0.229587
std	0.056021
min	0.102237
25%	0.191788
50%	0.216122
75%	0.255089
max	0.442010

In [ ]:
```
#Boxplot
import seaborn as sns
import matplotlib.pyplot as plt

#Plotar boxplot co2flux_600m
sns.boxplot(y='co2flux_600m', data = co2_BJ_val)

#Exportar figura
plt.savefig('co2flux_600m_boxplot.png', dpi=300, bbox_inches='tight')
```



In [ ]:
```
#Plotar histograma co2flux_600m
sns.histplot(co2_BJ_val, x = 'co2flux_600m', bins = 8, color = 'orange', kde = True, stat = 'probability')
```

Out[ ]: <Axes: xlabel='co2flux_600m', ylabel='Probability'>



In [ ]:
```
#Teste de distribuição normal dos dados de co2_BJ_val - TESTE LILLIEFORS (COLMOGOROV-SMINORV)
import statsmodels
from statsmodels.stats.diagnostic import lilliefors

statsmodels.stats.diagnostic.lilliefors(co2_BJ_val.co2flux_600m, dist = 'norm')
#O valor de p é 0.0009999999999998899, MENOR que 0,05. NÃO É DISTRIBUIÇÃO NORMAL
#O valor de p será sempre o segundo na lista (0.19863475951967197, p - > 0.2186282575697182)
#O primeiro valor é chamado de "estatística de teste"
```

Out[ ]: (0.12521525288339597, 0.0009999999999998899)

DADOS DE ESTIMATIVA DE FLUXO DE CO2 - SUPERFÍCIE

In [ ]:
```
#Importar bibliotecas
import pandas as pd
import geopandas as gpd
import os

#Importar dados das amostras de co2flux
#Criar variáveis referentes a cada planilha
co2_superficie = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ\\co2_superficie.csv", sep = ',', encoding = 'utf-8')

#Renomear coluna
co2_superficie = co2_superficie.rename(columns ={'first': 'co2flux_superficie'})

#Selecionar apenas a coluna com valores das amostras de co2flux
co2_superficie_val = co2_superficie['co2flux_superficie']

#Coverter coluna para dataframe
co2_superficie_val = pd.DataFrame(co2_superficie_val)

#Excluir valores NaN (nulos)
co2_superficie_val = co2_superficie_val.dropna()

#Análise de valores missing
co2_superficie_val.isnull().sum()
```

Out[ ]:
```
co2flux_superficie    0
dtype: int64
```

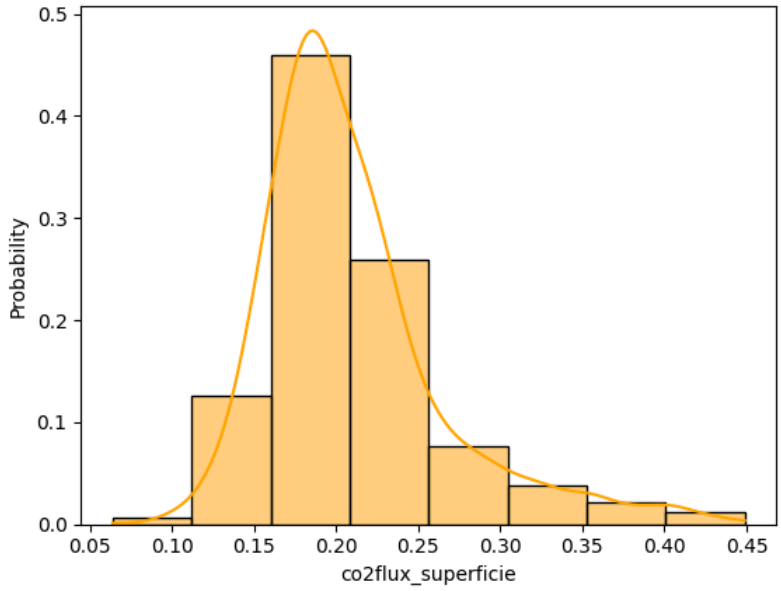
In [ ]:
```
co2_superficie_val.describe()
```

Out[ ]:

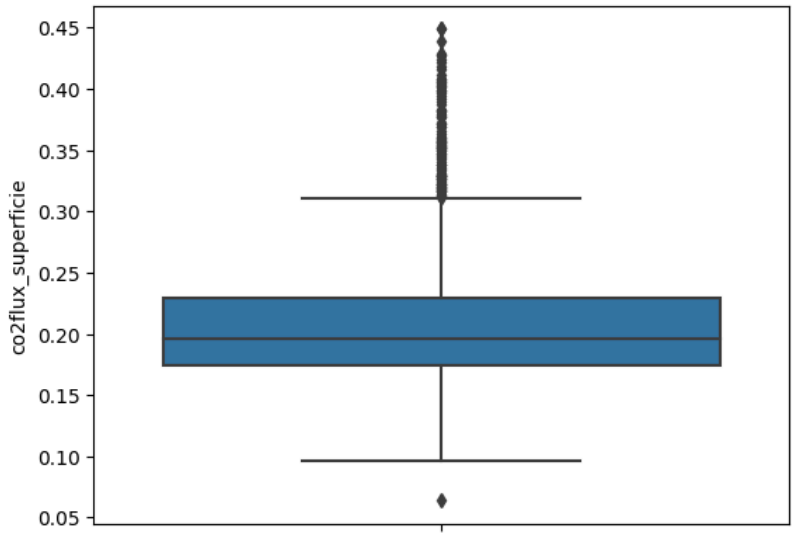
	co2flux_superficie
count	1691.000000
mean	0.209685
std	0.055783
min	0.063913
25%	0.174583
50%	0.196484
75%	0.229226
max	0.448912

In [ ]:
```
#Plotar histograma co2flux_superficie
sns.histplot(co2_superficie_val, x = 'co2flux_superficie', bins = 8, color = 'orange', kde = True, stat = 'probability')
```

Out[ ]:
```
<Axes: xlabel='co2flux_superficie', ylabel='Probability'>
```



In [ ]:
```
#Boxplot
import seaborn as sns
import matplotlib.pyplot as plt

#Plotar boxplot co2flux_superficie
sns.boxplot(y='co2flux_superficie', data = co2_superficie_val)

#Exportar figura
plt.savefig('co2flux_superficie_boxplot.png', dpi=300, bbox_inches='tight')
```



In [ ]:
```
#Teste de distribuição normal dos dados de co2_superficie_val - TESTE LILLIEFORS (COLMOGOROV-SMINORV)
import statsmodels
from statsmodels.stats.diagnostic import lilliefors

statsmodels.stats.diagnostic.lilliefors(co2_superficie_val.co2flux_superficie, dist = 'norm')
#O valor de p é 0.0009999999999998899, MENOR que 0,05. NÃO É DISTRIBUIÇÃO NORMAL
```

```
#O valor de p será sempre o segundo na lista (0.19863475951967197, p - > 0.2186282575697182)
#O primeiro valor é chamado de "estatística de teste"
```

Out[ ]: (0.1133743210101883, 0.0009999999999998899)

TRATAMENTOS DOS DADOS - ALERTAS DE INCÊNDIOS

ALERTAS PARA ÁREAS QUE NÃO SÃO SERRAS

In [ ]:
```
#Importar bibliotecas
import pandas as pd
import geopandas as gpd
import os

#Importar dados de alertas de incêncios/queimadas do Brasil entre 2012 e 2021
#Criar variáveis referentes a cada planilha
alerts_2012 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2012_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2013 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2013_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2014 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2014_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2015 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2015_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2016 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2016_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2017 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2017_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2018 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2018_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2019 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2019_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2020 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2020_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2021 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2021_Brazil.csv", sep = ',', encoding = 'utf-8')
```

```
C:\Users\Alisson Medeiros\AppData\Local\Temp\ipykernel_3140\3768031109.py:16: DtypeWarning: Columns (13) have mixed types. Specify dtype optio
n on import or set low_memory=False.
  alerts_2020 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2020_Brazil.csv", sep = ',', encoding = 'utf-8')
```

In [ ]:
```
#Converter alertas de 2012 para GeoDataFrame
gdf_2012 = gpd.GeoDataFrame(alerts_2012, geometry=gpd.points_from_xy(alerts_2012.longitude, alerts_2012.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

# Importar arquivo shp dos brejos de altitude
NE_set = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\recorte_dados.shp")

# Converter shp dos brejos de altitude para EPSG:4326
shape_NE_set = NE_set.to_crs("EPSG:4326")

#Filtrar alertas dentro dos brejos de altitude
alertas_NE_set_2012 = gpd.sjoin(gdf_2012, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2012.to_csv("alertas_NE_set_2012.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2013 para GeoDataFrame
gdf_2013 = gpd.GeoDataFrame(alerts_2013, geometry=gpd.points_from_xy(alerts_2013.longitude, alerts_2013.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2013 = gpd.sjoin(gdf_2013, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2013.to_csv("alertas_NE_set_2013.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2014 para GeoDataFrame
gdf_2014 = gpd.GeoDataFrame(alerts_2014, geometry=gpd.points_from_xy(alerts_2014.longitude, alerts_2014.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2014 = gpd.sjoin(gdf_2014, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2014.to_csv("alertas_NE_set_2014.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2015 para GeoDataFrame
gdf_2015 = gpd.GeoDataFrame(alerts_2015, geometry=gpd.points_from_xy(alerts_2015.longitude, alerts_2015.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2015 = gpd.sjoin(gdf_2015, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2015.to_csv("alertas_NE_set_2015.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2016 para GeoDataFrame
gdf_2016 = gpd.GeoDataFrame(alerts_2016, geometry=gpd.points_from_xy(alerts_2016.longitude, alerts_2016.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2016 = gpd.sjoin(gdf_2016, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2016.to_csv("alertas_NE_set_2016.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2017 para GeoDataFrame
gdf_2017 = gpd.GeoDataFrame(alerts_2017, geometry=gpd.points_from_xy(alerts_2017.longitude, alerts_2017.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2017 = gpd.sjoin(gdf_2017, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2017.to_csv("alertas_NE_set_2017.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2018 para GeoDataFrame
gdf_2018 = gpd.GeoDataFrame(alerts_2018, geometry=gpd.points_from_xy(alerts_2018.longitude, alerts_2018.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2018 = gpd.sjoin(gdf_2018, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2018.to_csv("alertas_NE_set_2018.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2019 para GeoDataFrame
gdf_2019 = gpd.GeoDataFrame(alerts_2019, geometry=gpd.points_from_xy(alerts_2019.longitude, alerts_2019.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2019 = gpd.sjoin(gdf_2019, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2019.to_csv("alertas_NE_set_2019.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2020 para GeoDataFrame
gdf_2020 = gpd.GeoDataFrame(alerts_2020, geometry=gpd.points_from_xy(alerts_2020.longitude, alerts_2020.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2020 = gpd.sjoin(gdf_2020, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2020.to_csv("alertas_NE_set_2020.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2021 para GeoDataFrame
gdf_2021 = gpd.GeoDataFrame(alerts_2021, geometry=gpd.points_from_xy(alerts_2021.longitude, alerts_2021.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados no Nordeste setentrional

#Filtrar alertas dentro do Nordeste setentrional
alertas_NE_set_2021 = gpd.sjoin(gdf_2021, shape_NE_set, op = 'within')

#Exportar o arquivo como .csv
alertas_NE_set_2021.to_csv("alertas_NE_set_2021.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```python
#Concatenar planilhas
import pandas as pd
import os
import geopandas as gpd

#Migrar para a pasta onde se encontram os arquivos .csv
os.chdir("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_NE_set")

#Verificar se estamos na pasta desejada (onde se encontram os arquivos .csv)
os.getcwd()

#Criar lista para armazenar os arquivos .csv
lista = []

#Salvar os conteúdos .csv em uma lista
for csv in os.listdir():
    df = pd.read_csv(csv)
    lista.append(df)

#Verificar arquivos da lista
lista

#Concatenar conteúdos
all_plan = pd.concat(lista)

#Verificar concatenação
all_plan

#Adcionar colunas "ano, "mes" e "dia"
all_plan[['Ano', 'Mês', 'Dia']] = all_plan['acq_date'].str.split('-', expand=True)

#Exportar o arquivo como .csv
all_plan.to_csv("alertas_NET_set_2012_2021.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\AppData\Local\Temp\ipykernel_3140\1179296447.py:17: DtypeWarning: Columns (13) have mixed types. Specify dtype optio
n on import or set low_memory=False.
  df = pd.read_csv(csv)
```

In [ ]:
```python
#Importar bibliotecas
import pandas as pd
import os

#Importar dataframe
all_plan = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_NE_set\\alertas_NET_set_2012_2021.csv")

#Selecionar as principais variáveis
var_prin_NE_set = all_plan.loc[:,['Ano', 'Mês', 'Dia']]
var_prin_NE_set
```

Out[ ]:

	Ano	Mês	Dia
0	2012	1	20
1	2012	1	20
2	2012	1	20
3	2012	1	20
4	2012	1	20
...	...	...	...
472769	2021	12	31
472770	2021	12	31
472771	2021	12	31
472772	2021	12	31
472773	2021	12	31

472774 rows × 3 columns

ALERTAS NOS BREJOS DE ALTITUDE

In [ ]:
```python
#Importar bibliotecas
import pandas as pd
import geopandas as gpd
import os

#Importar dados de alertas de incêncios/queimadas do Brasil entre 2012 e 2021
#Criar variáveis referentes a cada planilha
alerts_2012 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2012_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2013 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2013_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2014 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2014_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2015 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2015_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2016 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2016_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2017 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2017_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2018 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2018_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2019 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2019_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2020 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2020_Brazil.csv", sep = ',', encoding = 'utf-8')
alerts_2021 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2021_Brazil.csv", sep = ',', encoding = 'utf-8')
```

```
C:\Users\Alisson Medeiros\AppData\Local\Temp\ipykernel_3924\3768031109.py:16: DtypeWarning: Columns (13) have mixed types. Specify dtype optio
n on import or set low_memory=False.
  alerts_2020 = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\alertas_bra\\viirs-snpp_2020_Brazil.csv", sep = ',', encoding = 'utf-8')
```

In [ ]:
```
#Converter alertas de 2012 para GeoDataFrame
gdf_2012 = gpd.GeoDataFrame(alerts_2012, geometry=gpd.points_from_xy(alerts_2012.longitude, alerts_2012.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados os brejos de altitude

# Importar arquivo shp dos brejos de altitude
BJ_pol = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\600m_indiv.shp")

# Converter shp dos brejos de altitude para EPSG:4326
shape_BJ = BJ_pol.to_crs("EPSG:4326")

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2012 = gpd.sjoin(gdf_2012, shape_BJ, op = 'within')

#Exportar o arquivo como .csv
alertas_BJ_2012.to_csv("alertas_BJ_2012.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2013 para GeoDataFrame
gdf_2013 = gpd.GeoDataFrame(alerts_2013, geometry=gpd.points_from_xy(alerts_2013.longitude, alerts_2013.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2013 = gpd.sjoin(gdf_2013, shape_BJ, op = 'within')

#Exportar o arquivo como .csv
alertas_BJ_2013.to_csv("alertas_BJ_2013.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2014 para GeoDataFrame
gdf_2014 = gpd.GeoDataFrame(alerts_2014, geometry=gpd.points_from_xy(alerts_2014.longitude, alerts_2014.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2014 = gpd.sjoin(gdf_2014, shape_BJ, op = 'within')

#Exportar o arquivo como .csv
alertas_BJ_2014.to_csv("alertas_BJ_2014.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
# Converter alertas de 2015 para GeoDataFrame
gdf_2015 = gpd.GeoDataFrame(alerts_2015, geometry=gpd.points_from_xy(alerts_2015.longitude, alerts_2015.latitude), crs="EPSG:4326")

# Selecionar APENAS os alertas registrados nos brejos de altitude

# Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2015 = gpd.sjoin(gdf_2015, shape_BJ, op='within')

# Exportar o arquivo como .csv
alertas_BJ_2015.to_csv("alertas_BJ_2015.csv", sep=',', encoding='utf-8', index=False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2016 para GeoDataFrame
gdf_2016 = gpd.GeoDataFrame(alerts_2016, geometry=gpd.points_from_xy(alerts_2016.longitude, alerts_2016.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2016 = gpd.sjoin(gdf_2016, shape_BJ, op = 'within')

#Exportar o arquivo como .csv
alertas_BJ_2016.to_csv("alertas_BJ_2016.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2017 para GeoDataFrame
gdf_2017 = gpd.GeoDataFrame(alerts_2017, geometry=gpd.points_from_xy(alerts_2017.longitude, alerts_2017.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2017 = gpd.sjoin(gdf_2017, shape_BJ, op = 'within')
```

```
#Exportar o arquivo como .csv
alertas_BJ_2017.to_csv("alertas_BJ_2017.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2018 para GeoDataFrame
gdf_2018 = gpd.GeoDataFrame(alerts_2018, geometry=gpd.points_from_xy(alerts_2018.longitude, alerts_2018.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2018 = gpd.sjoin(gdf_2018, shape_BJ, op = 'within')

#Exportar o arquivo como .csv
alertas_BJ_2018.to_csv("alertas_BJ_2018.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2019 para GeoDataFrame
gdf_2019 = gpd.GeoDataFrame(alerts_2019, geometry=gpd.points_from_xy(alerts_2019.longitude, alerts_2019.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2019 = gpd.sjoin(gdf_2019, shape_BJ, op='within')

#Exportar o arquivo como .csv
alertas_BJ_2019.to_csv("alertas_BJ_2019.csv", sep=',', encoding='utf-8', index=False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2020 para GeoDataFrame
gdf_2020 = gpd.GeoDataFrame(alerts_2020, geometry=gpd.points_from_xy(alerts_2020.longitude, alerts_2020.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2020 = gpd.sjoin(gdf_2020, shape_BJ, op = 'within')

#Exportar o arquivo como .csv
alertas_BJ_2020.to_csv("alertas_BJ_2020.csv", sep=',', encoding='utf-8', index = False)
```

```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Converter alertas de 2021 para GeoDataFrame
gdf_2021 = gpd.GeoDataFrame(alerts_2021, geometry=gpd.points_from_xy(alerts_2021.longitude, alerts_2021.latitude), crs="EPSG:4326")

#Selecionar APENAS os alertas registrados nos brejos de altitude

#Filtrar alertas dentro dos brejos de altitude
alertas_BJ_2021 = gpd.sjoin(gdf_2021, shape_BJ, op = 'within')

#Exportar o arquivo como .csv
alertas_BJ_2021.to_csv("alertas_BJ_2021.csv", sep=',', encoding='utf-8', index = False)
```

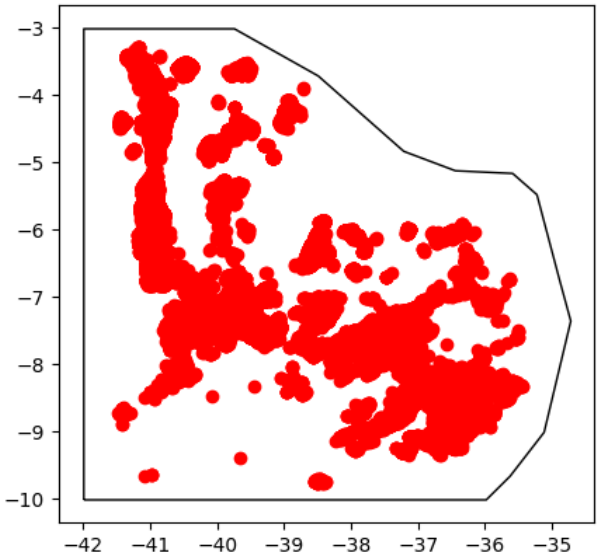
```
C:\Users\Alisson Medeiros\anaconda3\envs\Phytho_GIS\lib\site-packages\IPython\core\interactiveshell.py:3445: FutureWarning: The `op` parameter
is deprecated and will be removed in a future release. Please use the `predicate` parameter instead.
  if await self.run_code(code, result, async_=asy):
```

In [ ]:
```
#Concatenar planilhas
import pandas as pd
import os
import geopandas as gpd

#Migrar para a pasta onde se encontram os arquivos .csv
os.chdir("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ")

#Verificar se estamos na pasta desejada (onde se encontram os arquivos .csv)
os.getcwd()

#Criar lista para armazenar os arquivos .csv
lista = []

#Salvar os conteúdos .csv em uma lista
for csv in os.listdir():
    df = pd.read_csv(csv)
    lista.append(df)

#Verificar arquivos da lista
lista

#Concatenar conteúdos
all_plan = pd.concat(lista)

#Verificar concatenação
all_plan
```

```python
#Adcionar colunas "ano, "mes" e "dia"
all_plan[['Ano', 'Mês', 'Dia']] = all_plan['acq_date'].str.split('-', expand=True)

#Exportar o arquivo como .csv
all_plan.to_csv("alertas_BJ_2012_2021.csv", sep=',', encoding='utf-8', index = False)

#Converter alertas de 2021 para GeoDataFrame
all_plan_gdf = gpd.GeoDataFrame(all_plan, geometry=gpd.points_from_xy(all_plan.longitude, all_plan.latitude), crs="EPSG:4326")

#Exportar geodataframe como shp
all_plan_gdf.to_file('C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ\\alertas_BJ_2012_2021.shp')
```

In [ ]:

```python
#Visalizar pontos

#Importar bibliotecas
import geopandas as gpd
import matplotlib.pyplot as plt
from geodatasets import get_path

#Importar "mundo"
world = gpd.read_file(get_path("naturalearth.land"))

# Restringir para a área de estudo
ax = world.clip([-42, -10, -30, -3]).plot(color="white", edgecolor="black")

# Plotar pontos com base no ``GeoDataFrame``.
all_plan_gdf.plot(ax=ax, color="red")

#Mostrar pontos
plt.show()
```



In [ ]:

```python
#Importar bibliotecas
import pandas as pd
import os
#Importar dataframe
all_plan = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ\\alertas_BJ_2012_2021.csv")

#Selecionar as principais variáveis
var_prin = all_plan.loc[:,['Ano', 'Mês', 'Dia']]
var_prin
```

Out[ ]:

	Ano	Mês	Dia
0	2012	1	20
1	2012	1	30
2	2012	1	30
3	2012	2	1
4	2012	2	3
...	...	...	...
62772	2021	12	17
62773	2021	12	19
62774	2021	12	23
62775	2021	12	25
62776	2021	12	29

62777 rows × 3 columns

In [ ]:

```python
#Relação alertas serras / Nordeste setentrional
bj = 62777
```

```
ne_set_tot = 472774
alertas_bj_porcento = (bj/ne_set_tot)*100

print(alertas_bj_porcento)
```

```
13.278437477526259
```

In [ ]:
```
#Selecionar total de alertas por ano (2012)
alertas_2012 = var_prin[var_prin['Ano'] == 2012]
alertas_2012
```

Out[ ]:

	Ano	Mês	Dia
0	2012	1	20
1	2012	1	30
2	2012	1	30
3	2012	2	1
4	2012	2	3
...	...	...	...
6232	2012	12	24
6233	2012	12	24
6234	2012	12	26
6235	2012	12	29
6236	2012	12	29

6237 rows × 3 columns

In [ ]:
```
# Selecionar total de alertas por ano (2013)
alertas_2013 = var_prin[var_prin['Ano'] == 2013]
alertas_2013
```

Out[ ]:

	Ano	Mês	Dia
6237	2013	1	1
6238	2013	1	1
6239	2013	1	1
6240	2013	1	1
6241	2013	1	1
...	...	...	...
11371	2013	12	26
11372	2013	12	28
11373	2013	12	28
11374	2013	12	28
11375	2013	12	31

5139 rows × 3 columns

In [ ]:
```
# Selecionar total de alertas por ano (2014)
alertas_2014 = var_prin[var_prin['Ano'] == 2014]
alertas_2014
```

Out[ ]:

	Ano	Mês	Dia
11376	2014	1	1
11377	2014	1	1
11378	2014	1	1
11379	2014	1	1
11380	2014	1	1
...	...	...	...
16045	2014	12	30
16046	2014	12	20
16047	2014	12	20
16048	2014	12	22
16049	2014	12	27

4674 rows × 3 columns

In [ ]:
```
# Selecionar total de alertas por ano (2015)
alertas_2015 = var_prin[var_prin['Ano'] == 2015]
alertas_2015
```

Out[ ]:

	Ano	Mês	Dia
16050	2015	1	1
16051	2015	1	3
16052	2015	1	3
16053	2015	1	6
16054	2015	1	7
...	...	...	...
21322	2015	12	21
21323	2015	12	22
21324	2015	12	25
21325	2015	12	27
21326	2015	12	29

5277 rows × 3 columns

In [ ]:
```
# Selecionar total de alertas por ano (2016)
alertas_2016 = var_prin[var_prin['Ano'] == 2016]
alertas_2016
```

Out[ ]:

	Ano	Mês	Dia
21327	2016	1	1
21328	2016	1	1
21329	2016	1	2
21330	2016	1	2
21331	2016	1	2
...	...	...	...
28102	2016	12	26
28103	2016	12	27
28104	2016	12	30
28105	2016	12	30
28106	2016	12	31

6780 rows × 3 columns

In [ ]:
```
# Selecionar total de alertas por ano (2017)
alertas_2017 = var_prin[var_prin['Ano'] == 2017]
alertas_2017
```

Out[ ]:

	Ano	Mês	Dia
28107	2017	1	1
28108	2017	1	3
28109	2017	1	4
28110	2017	1	4
28111	2017	1	4
...	...	...	...
33738	2017	12	26
33739	2017	12	27
33740	2017	12	29
33741	2017	12	29
33742	2017	12	30

5636 rows × 3 columns

In [ ]:
```
# Selecionar total de alertas por ano (2018)
alertas_2018 = var_prin[var_prin['Ano'] == 2018]
alertas_2018
```

Out[ ]:

	Ano	Mês	Dia
33743	2018	1	1
33744	2018	1	2
33745	2018	1	3
33746	2018	1	3
33747	2018	1	4
...	...	...	...
39003	2018	12	20
39004	2018	12	28
39005	2018	12	28
39006	2018	12	29
39007	2018	12	31

5265 rows × 3 columns

In [ ]:

```
# Selecionar total de alertas por ano (2019)
alertas_2019 = var_prin[var_prin['Ano'] == 2019]
alertas_2019
```

Out[ ]:

	Ano	Mês	Dia
39008	2019	1	1
39009	2019	1	1
39010	2019	1	1
39011	2019	1	1
39012	2019	1	1
...	...	...	...
46499	2019	12	21
46500	2019	12	22
46501	2019	12	22
46502	2019	12	23
46503	2019	12	27

7496 rows × 3 columns

In [ ]:

```
# Selecionar total de alertas por ano (2020)
alertas_2020 = var_prin[var_prin['Ano'] == 2020]
alertas_2020
```

Out[ ]:

	Ano	Mês	Dia
46504	2020	1	3
46505	2020	1	3
46506	2020	1	13
46507	2020	1	13
46508	2020	1	13
...	...	...	...
54095	2020	12	26
54096	2020	12	28
54097	2020	12	29
54098	2020	12	30
54099	2020	12	30

7596 rows × 3 columns

In [ ]:

```
# Selecionar total de alertas por ano (2021)
alertas_2021 = var_prin[var_prin['Ano'] == 2021]
alertas_2021
```

Out[ ]:

	Ano	Mês	Dia
54100	2021	1	1
54101	2021	1	1
54102	2021	1	1
54103	2021	1	1
54104	2021	1	2
...	...	...	...
62772	2021	12	17
62773	2021	12	19
62774	2021	12	23
62775	2021	12	25
62776	2021	12	29

8677 rows × 3 columns

In [ ]:

```
#Criar dataframe com número de queimadas por ano
df_alerts_ano = pd.DataFrame({
    "Ano": [2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021],
    "Total_alertas_ano": [6237, 5139, 4674, 5277, 6780, 5636, 5265, 7496, 7596, 8677],
})
df_alerts_ano

#Exportar dataframe
df_alerts_ano.to_csv("df_alerts_ano.csv", sep=',', encoding='utf-8', index = False)
```

In [ ]:

```
#Número total de alertas por mês
alertas_jan = var_prin[var_prin['Mês'] == 1]
alertas_fev = var_prin[var_prin['Mês'] == 2]
alertas_mar = var_prin[var_prin['Mês'] == 3]
alertas_abr = var_prin[var_prin['Mês'] == 4]
alertas_mai = var_prin[var_prin['Mês'] == 5]
alertas_jun = var_prin[var_prin['Mês'] == 6]
alertas_jul = var_prin[var_prin['Mês'] == 7]
alertas_ago = var_prin[var_prin['Mês'] == 8]
alertas_set = var_prin[var_prin['Mês'] == 9]
alertas_out = var_prin[var_prin['Mês'] == 10]
alertas_nov = var_prin[var_prin['Mês'] == 11]
alertas_dez = var_prin[var_prin['Mês'] == 12]
```

In [ ]:

```
print(alertas_jan,
      alertas_fev,
      alertas_mar,
      alertas_abr,
      alertas_mai,
      alertas_jun,
      alertas_jul,
      alertas_ago,
      alertas_set,
      alertas_out,
      alertas_nov,
      alertas_dez)
```

```
        Ano  Mês  Dia
0      2012    1   20
1      2012    1   30
2      2012    1   30
31     2012    1   21
32     2012    1   21
...     ...  ...  ...
61920  2021    1   15
61921  2021    1   15
61922  2021    1   15
61923  2021    1   15
61924  2021    1   15

[3583 rows x 3 columns]         Ano  Mês  Dia
3      2012    2    1
4      2012    2    3
5      2012    2    4
6      2012    2    4
7      2012    2    4
...     ...  ...  ...
61985  2021    2   20
61986  2021    2   21
61987  2021    2   25
61988  2021    2   25
61989  2021    2   25

[1956 rows x 3 columns]         Ano  Mês  Dia
12     2012    3   17
403    2012    3    1
404    2012    3    1
405    2012    3    1
406    2012    3    1
...     ...  ...  ...
61999  2021    3    9
62000  2021    3    9
62001  2021    3   16
62002  2021    3   20
62003  2021    3   20

[1192 rows x 3 columns]         Ano  Mês  Dia
13     2012    4   19
14     2012    4   19
666    2012    4    1
667    2012    4    1
668    2012    4    1
...     ...  ...  ...
61971  2021    4    3
61974  2021    4    6
61975  2021    4   14
62004  2021    4    1
62005  2021    4   27

[770 rows x 3 columns]         Ano  Mês  Dia
15     2012    5    1
875    2012    5    1
876    2012    5    1
877    2012    5    1
878    2012    5    1
...     ...  ...  ...
59685  2021    5   12
59686  2021    5   29
61310  2021    5   26
61848  2021    5   22
61849  2021    5   22

[472 rows x 3 columns]         Ano  Mês  Dia
981    2012    6    1
982    2012    6    1
983    2012    6    1
984    2012    6    1
985    2012    6    1
...     ...  ...  ...
61722  2021    6   18
61850  2021    6   12
61851  2021    6   12
62006  2021    6   14
62007  2021    6   29

[406 rows x 3 columns]         Ano  Mês  Dia
1020   2012    7    1
1021   2012    7    4
1022   2012    7   10
1023   2012    7   10
1024   2012    7   10
...     ...  ...  ...
61452  2021    7   28
61883  2021    7   21
62042  2021    7   17
62043  2021    7   17
62062  2021    7   30

[646 rows x 3 columns]         Ano  Mês  Dia
16     2012    8   15
17     2012    8   22
```

```
18      2012    8   24
1041    2012    8    2
1042    2012    8    3
...      ...  ...  ...
62159   2021    8   29
62160   2021    8   29
62161   2021    8   29
62162   2021    8   30
62163   2021    8   29

[2246 rows x 3 columns]         Ano  Mês  Dia
19      2012    9    4
20      2012    9   17
21      2012    9   17
22      2012    9   17
23      2012    9   24
...      ...  ...  ...
62201   2021    9   30
62202   2021    9   26
62203   2021    9   28
62204   2021    9   28
62205   2021    9   28

[6352 rows x 3 columns]         Ano  Mês  Dia
24      2012   10    5
25      2012   10   12
1195    2012   10    1
1196    2012   10    1
1197    2012   10    2
...      ...  ...  ...
62654   2021   10   29
62655   2021   10   30
62656   2021   10   31
62657   2021   10   31
62658   2021   10   31

[16790 rows x 3 columns]         Ano  Mês  Dia
26      2012   11    2
27      2012   11    8
28      2012   11    8
1413    2012   11    1
1414    2012   11    1
...      ...  ...  ...
62747   2021   11   26
62751   2021   11   27
62752   2021   11   30
62753   2021   11   30
62754   2021   11   30

[16690 rows x 3 columns]         Ano  Mês  Dia
29      2012   12    3
30      2012   12    8
1728    2012   12    1
1729    2012   12    1
1730    2012   12    1
...      ...  ...  ...
62772   2021   12   17
62773   2021   12   19
62774   2021   12   23
62775   2021   12   25
62776   2021   12   29

[11674 rows x 3 columns]
```

In [ ]:
```
#Criar dataframe com número de queimadas por mês
df_alerts_mês = pd.DataFrame({
    "Mês": ['Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio', 'Junho', 'Julho', 'Agosto', 'Setembro', 'Outubro', 'Novembro', 'Dezembro'],
    "Total_alertas_mês": [3583, 1956, 1192, 770, 472, 406, 646, 2246, 6352, 16790, 16690, 11674]
})
df_alerts_mês

#Exportar dataframe
df_alerts_mês.to_csv("df_alerts_mês.csv", sep=',', encoding='utf-8', index = False)
```

ANÁLISE ESTATÍSTICA

BJ - ANÁLISE ANUAL E MENSAL

In [ ]:
```
import pandas as pd
import geopandas as gpd
import os

#Importar planilha alertas/ano RN
alertas_BJ_ano = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ\\df_alerts_ano.csv", sep = ',', encoding = 'utf-8')

#Importar planilha alertas/mes RN
alertas_BJ_mes = pd.read_csv("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ\\df_alerts_mês.csv", sep = ',', encoding = 'utf-8')
```

In [ ]:
```
alertas_BJ_mes
```

Out[ ]:

	Mês	Total_alertas_mês
0	Janeiro	3583
1	Fevereiro	1956
2	Março	1192
3	Abril	770
4	Maio	472
5	Junho	406
6	Julho	646
7	Agosto	2246
8	Setembro	6352
9	Outubro	16790
10	Novembro	16690
11	Dezembro	11674

In [ ]:
```
alertas_BJ_ano
```

Out[ ]:

	Ano	Total_alertas_ano
0	2012	6237
1	2013	5139
2	2014	4674
3	2015	5277
4	2016	6780
5	2017	5636
6	2018	5265
7	2019	7496
8	2020	7596
9	2021	8677

In [ ]:
```
#Total de alertas no período de 2012 a 2021
total_alerts_BJ = sum(alertas_BJ_ano.Total_alertas_ano)
total_alerts_BJ
```

Out[ ]: 62777

In [ ]:
```
#Estatistica descritiva ano
alertas_BJ_ano.describe()
```

Out[ ]:

	Ano	Total_alertas_ano
count	10.00000	10.000000
mean	2016.50000	6277.700000
std	3.02765	1314.027063
min	2012.00000	4674.000000
25%	2014.25000	5268.000000
50%	2016.50000	5936.500000
75%	2018.75000	7317.000000
max	2021.00000	8677.000000

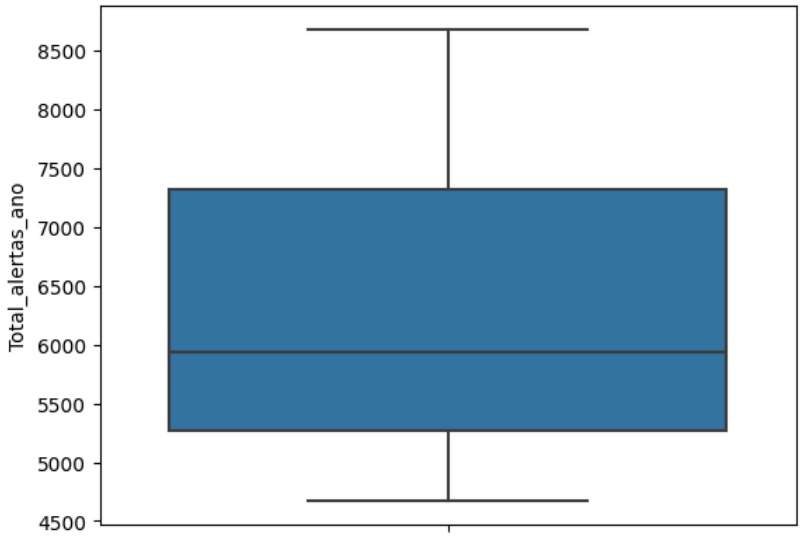
In [ ]:
```
#Estatistica descritiva ano
alertas_BJ_mes.describe()
```

Out[ ]:

	Total_alertas_mês
count	12.000000
mean	5231.416667
std	6275.528061
min	406.000000
25%	739.000000
50%	2101.000000
75%	7682.500000
max	16790.000000

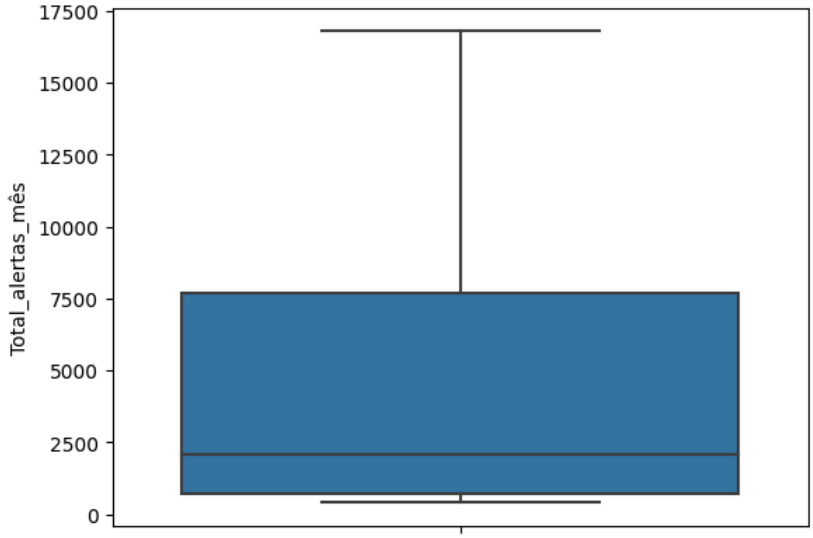
In [ ]:
```
#Boxplot
import seaborn as sns

#Plotar boxplot anual
sns.boxplot(y='Total_alertas_ano', data = alertas_BJ_ano)
```

Out[ ]: <Axes: ylabel='Total_alertas_ano'>



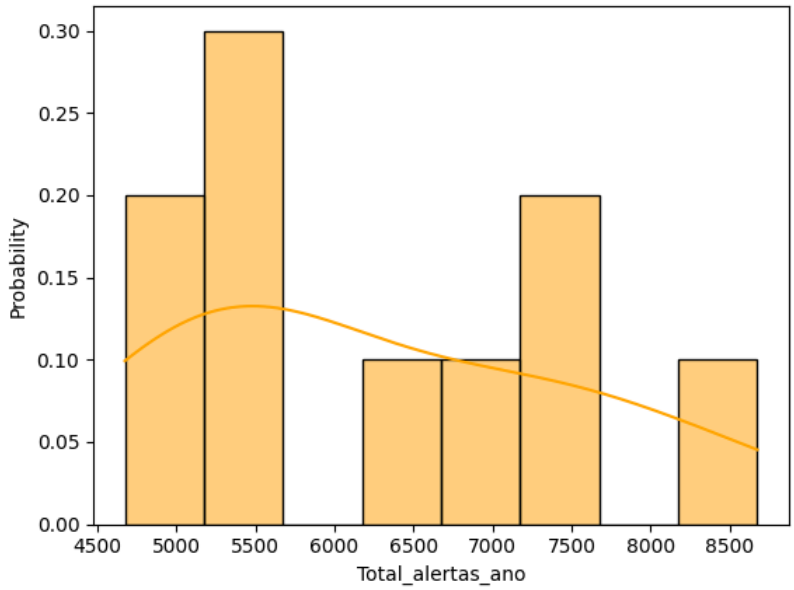
In [ ]:
```
#Boxplot
import seaborn as sns

#Plotar boxplot anual
sns.boxplot(y='Total_alertas_mês', data = alertas_BJ_mes)
```

Out[ ]: <Axes: ylabel='Total_alertas_mês'>



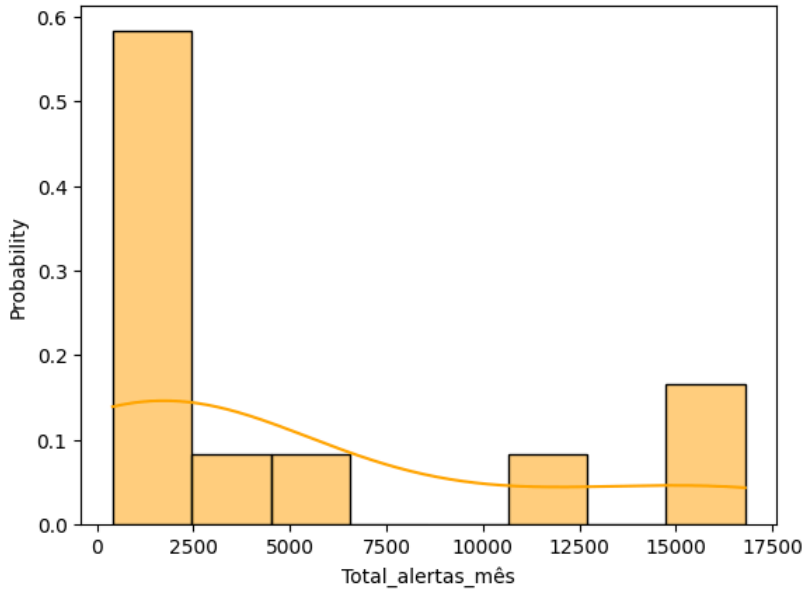
In [ ]:
```
#Plotar histograma alertas anuais
sns.histplot(alertas_BJ_ano, x = 'Total_alertas_ano', bins = 8, color = 'orange', kde = True, stat = 'probability')
```

Out[ ]: <Axes: xlabel='Total_alertas_ano', ylabel='Probability'>

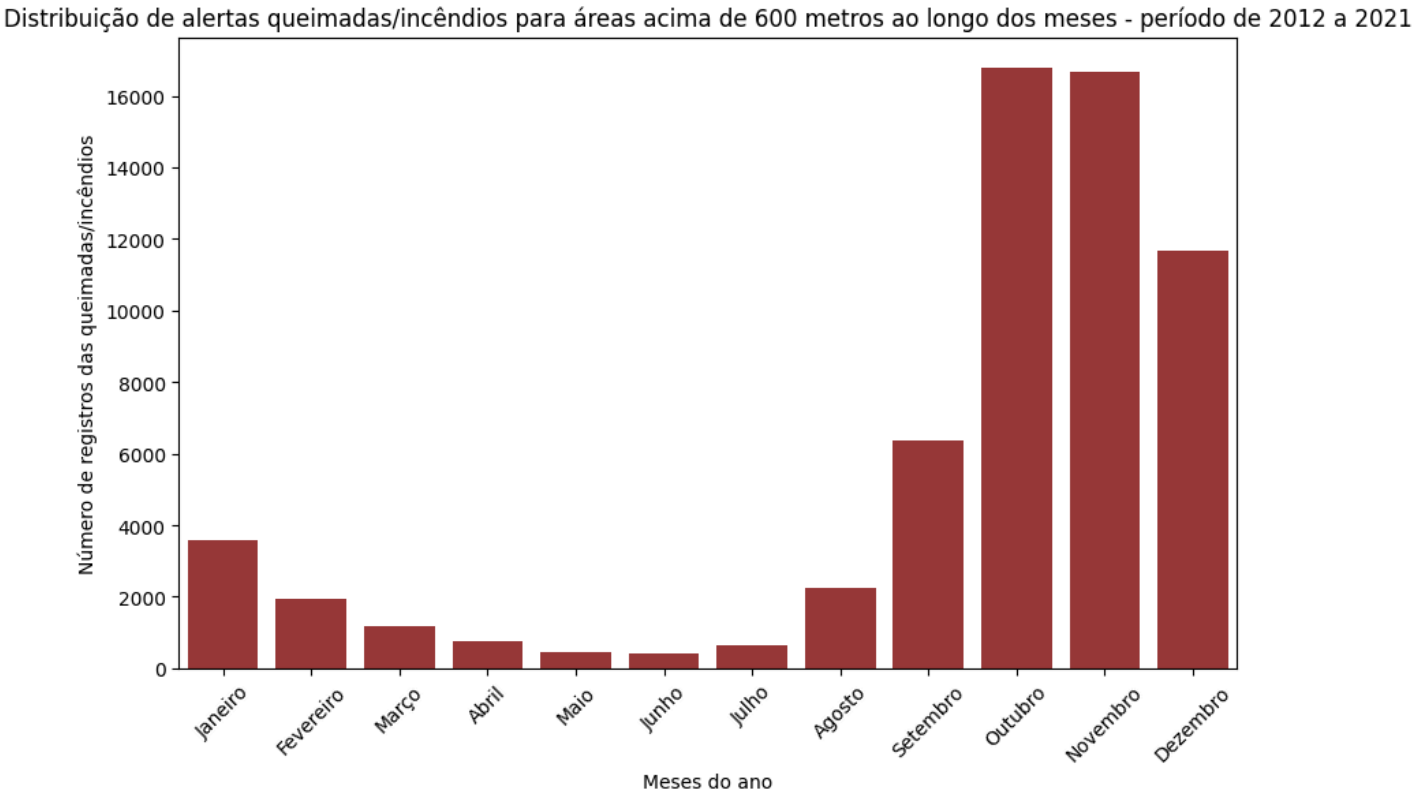


In [ ]:
```
#Plotar histograma alertas mensais
sns.histplot(alertas_BJ_mes, x = 'Total_alertas_mês', bins = 8, color = 'orange', kde = True, stat = 'probability')
```

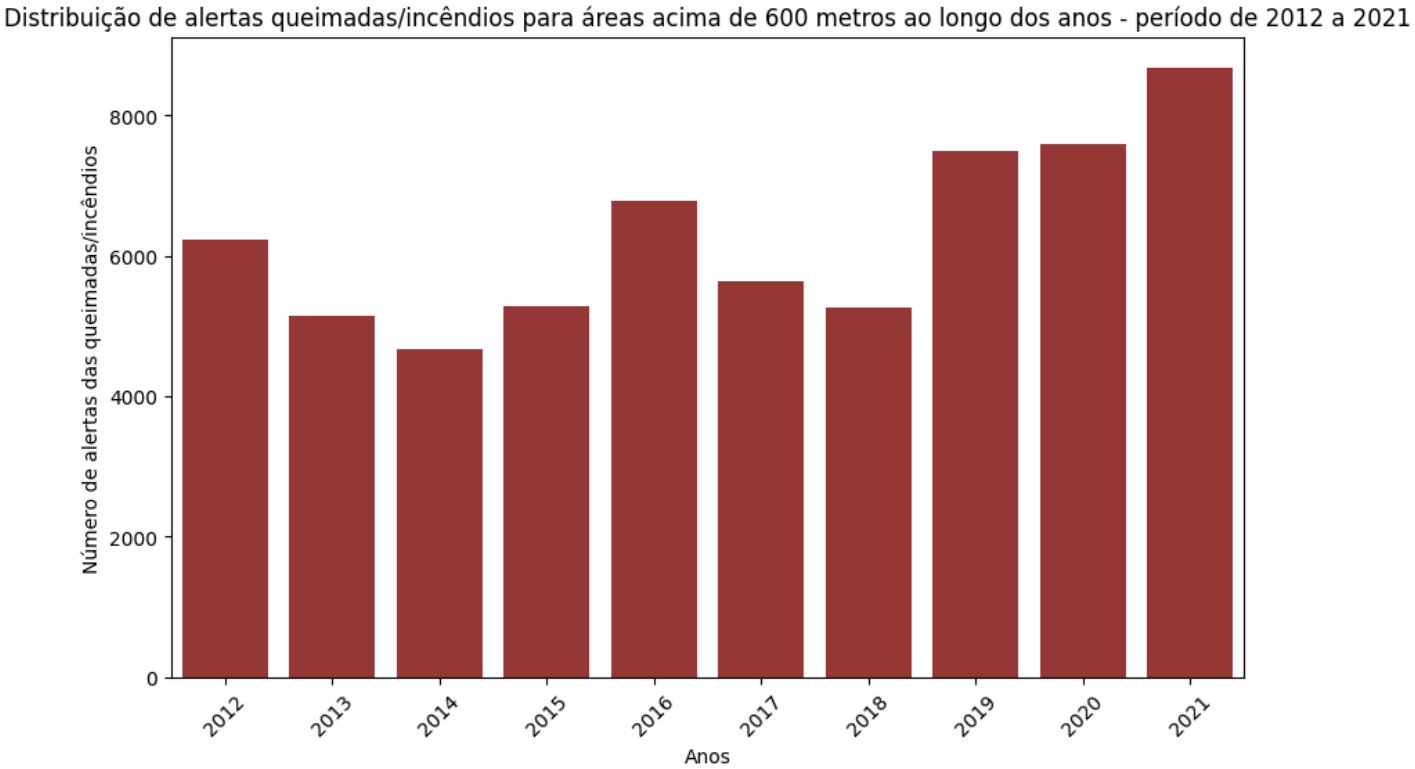
Out[ ]:
```
<Axes: xlabel='Total_alertas_mês', ylabel='Probability'>
```



In [ ]:
```
#Importar bibliotecas
import matplotlib.pyplot as plt
import seaborn as sns

#Gerar histpgrma de distribuição de queimadas ao longos dos meses do ano entre 2012 e 2021
plt.figure(figsize = (10,6))
sns.barplot(x = 'Mês', y = 'Total_alertas_mês',
            data = alertas_BJ_mes, color = "brown")
plt.xlabel('Meses do ano')
plt.ylabel('Número de registros das queimadas/incêndios')
plt.title('Distribuição de alertas queimadas/incêndios para áreas acima de 600 metros ao longo dos meses - período de 2012 a 2021')
plt.xticks(rotation = 45)

# Exportar a figura
plt.savefig('grafico_distribuição_mes_BJ_queimadas.png', dpi = 300, bbox_inches = 'tight')

#Mostrar gráfico
plt.show()
```



In [ ]:
```
#Gerar histpgrma de distribuição de queimadas ao longos dos anos entre 2012 e 2021
plt.figure(figsize = (10,6))
sns.barplot(x = 'Ano', y = 'Total_alertas_ano',
            data = alertas_BJ_ano, color = "brown")
plt.xlabel('Anos')
plt.ylabel('Número de alertas das queimadas/incêndios')
plt.title('Distribuição de alertas queimadas/incêndios para áreas acima de 600 metros ao longo dos anos - período de 2012 a 2021')
plt.xticks(rotation = 45)

# Exportar a figura
plt.savefig('grafico_distribuição_ano_BJ_queimadas.png', dpi = 300, bbox_inches = 'tight')

#Mostrar gráfico
plt.show()
```



In [ ]:
```
#Teste de distribuição normal dos dados de alertas anuais - TESTE LILLIEFORS (COLMOGOROV-SMINORV)
import statsmodels
from statsmodels.stats.diagnostic import lilliefors

statsmodels.stats.diagnostic.lilliefors(alertas_BJ_ano.Total_alertas_ano, dist = 'norm')
#O valor de p é 0.41677505210230503, MAIOR que 0,05. É DISTRIBUIÇÃO NORMAL
#O valor de p será sempre o segundo na lista (0.16174191924671233, p - > 0.6427448499952756)
#O primeiro valor é chamado de "estatística de teste"
```

Out[ ]: (0.18734764568313078, 0.41677505210230503)

In [ ]:
```
#Teste de distribuição normal dos dados de alertas mensais - TESTE LILLIEFORS (COLMOGOROV-SMINORV)
import statsmodels
from statsmodels.stats.diagnostic import lilliefors

statsmodels.stats.diagnostic.lilliefors(alertas_BJ_mes.Total_alertas_mês, dist = 'norm')
#O valor de p é 0.017441871716375432, MENOR que 0,05. NÃO É DISTRIBUIÇÃO NORMAL
#O valor de p será sempre o segundo na lista (0.19863475951967197, p - > 0.2186282575697182)
#O primeiro valor é chamado de "estatística de teste"
```

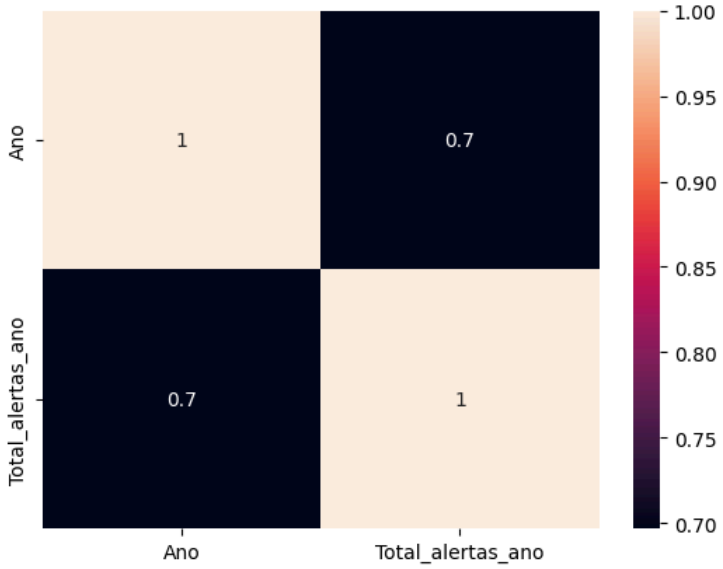
Out[ ]: (0.2702656520996609, 0.017441871716375432)

In [ ]:
```
#Correlação de Spearman
corrS = alertas_BJ_ano.corr(method = 'spearman')
corrS
```

Out[ ]:

	Ano	Total_alertas_ano
Ano	1.00000	0.69697
Total_alertas_ano	0.69697	1.00000

In [ ]:
```
import matplotlib.pyplot as plt

plt.figure()
sns.heatmap(corrS, annot = True);
```



ESTATÍSTICA ZONAL

In [ ]:
```
#Calcular pontos por polígonos

import geopandas as gpd

# Carregar o shapefile de polígonos e o shapefile de pontos
polygons = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\shp\\600m_indiv.shp")
points = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\csv_BJ\\alertas_BJ_2012_2021.shp")

# Garantir que ambos estão no mesmo sistema de coordenadas
if polygons.crs != points.crs:
    points = points.to_crs(polygons.crs)

# Criar uma nova coluna no GeoDataFrame de polígonos para contar os pontos
polygons["num_points"] = 0

# Iterar sobre os polígonos e contar quantos pontos estão dentro de cada um
for idx, poly in polygons.iterrows():
    polygons.loc[idx, "num_points"] = points.within(poly.geometry).sum()

# Ordenar os polígonos pelo número de pontos em ordem decrescente
polygons_sorted = polygons.sort_values("num_points", ascending=False)

# Exibir o resultado
print(polygons_sorted[["num_points", "geometry"]])

# Salvar o resultado em um novo shapefile, se necessário
polygons_sorted.to_file("poligonos_com_contagem_de_pontos.shp")
```

```
       num_points                                           geometry
11872       17445  POLYGON ((-36.71694 -9.26097, -36.71685 -9.261...
7966        12919  POLYGON ((-40.68417 -7.81514, -40.68459 -7.815...
4004        10349  POLYGON ((-40.86917 -6.83319, -40.86926 -6.832...
1080         5076  POLYGON ((-40.98750 -4.97375, -40.98685 -4.973...
971          2198  POLYGON ((-40.05111 -4.86153, -40.05145 -4.861...
...           ...                                                ...
4226            0  POLYGON ((-39.24278 -6.92181, -39.24373 -6.921...
4227            0  POLYGON ((-36.33778 -6.92181, -36.33768 -6.921...
4228            0  POLYGON ((-40.15583 -6.92208, -40.15623 -6.921...
4229            0  POLYGON ((-39.85306 -6.92208, -39.85361 -6.922...
6080            0  POLYGON ((-35.47194 -7.48375, -35.47343 -7.483...

[12161 rows x 2 columns]
```

In [ ]:

```python
#Calcular densidade de pontos por km2

#Importar shp
import geopandas as gpd
import pandas as pd

BJ_metros = gpd.read_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\poligonos_com_contagem_de_pontos.shp")

#Excluir polígonos com valores inferiores a "0" na coluna "num_points"

BJ_metros['num_points'] = pd.to_numeric(BJ_metros['num_points'], errors='coerce')

BJ_metros = BJ_metros[BJ_metros['num_points'] >= 0.1]

#Converter src para mercartor (epsg=3395)
BJ_metros = BJ_metros.to_crs(epsg=3395)

#Calcular área (m2 para km2)
BJ_metros['area_km2'] = BJ_metros['geometry'].area/1000000

#Calcular densidade
BJ_metros['Alertas_por_km2'] = BJ_metros['num_points']/BJ_metros['area_km2']
BJ_metros
#Salvar arquivo
BJ_metros.to_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\poligonos_com_contagem_de_pontos.shp")
```

```
C:\Users\Alisson Medeiros\AppData\Local\Temp\ipykernel_8536\1554570097.py:25: UserWarning: Column names longer than 10 characters will be trun
cated when saved to ESRI Shapefile.
  BJ_metros.to_file("C:\\Users\\Alisson Medeiros\\tese_cap3\\poligonos_com_contagem_de_pontos.shp")
```

In [ ]:

```python
#Estatistica descritiva ano
BJ_metros.describe()
```

Out[ ]:

	OBJECTID	Id	gridcode	Shape_Leng	Shape_Area	num_points	area_km2	Alertas_po	Alertas_por_km2
count	717.000000	717.000000	717.0	717.000000	7.170000e+02	717.000000	717.000000	717.000000	717.000000
mean	7033.873082	7033.873082	2.0	0.482627	5.698448e-03	87.555091	70.722279	13.365334	13.365334
std	5107.323488	5107.323488	0.0	4.024873	6.811192e-02	923.841446	846.481504	69.691305	69.691305
min	15.000000	15.000000	2.0	0.001048	5.278521e-08	1.000000	0.000655	0.025514	0.025514
25%	2760.000000	2760.000000	2.0	0.020450	1.710351e-05	1.000000	0.211982	0.996614	0.996614
50%	5980.000000	5980.000000	2.0	0.052563	7.225659e-05	2.000000	0.892257	2.744097	2.744097
75%	11632.000000	11632.000000	2.0	0.155298	3.798628e-04	6.000000	4.700820	8.279090	8.279090
max	16494.000000	16494.000000	2.0	98.809281	1.574742e+00	17445.000000	19587.847770	1525.884100	1525.884100

In [ ]:

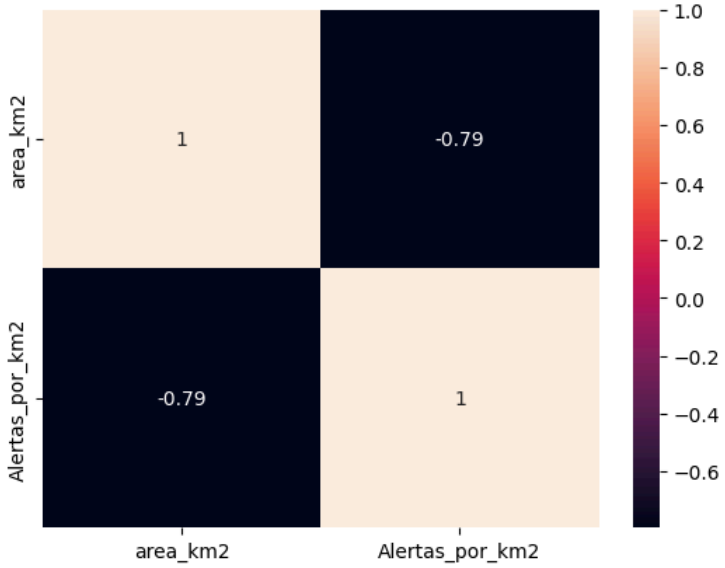
```python
#Selecionar colunas area_km2 e Alertas_por_km2
BJ_spearman = BJ_metros[['area_km2', 'Alertas_por_km2']]

#Correlação de Spearman
corrS = BJ_spearman.corr(method = 'spearman')
corrS

#Quanto maior a área, menor a densidade de alertas
```

Out[ ]:

	area_km2	Alertas_por_km2
area_km2	1.000000	-0.794786
Alertas_por_km2	-0.794786	1.000000

In [ ]:

```python
import matplotlib.pyplot as plt

plt.figure()
sns.heatmap(corrS, annot = True);
```


area_km2
Alertas_por_km2
1
-0.79
-0.79
1
area_km2
Alertas_por_km2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6